Lab 8 — Thống kê

1. Đề bài tiếng Anh

LAB SESSION 8

Problem Description

A mobile application company wants to analyze users' daily app usage time to understand usage patterns, data variability, and statistical characteristics.

You are given daily app usage time for **50 users**, measured in **minutes per day**.

Dataset

```
usage_minutes = [
    35, 45, 50, 55, 60, 65, 70, 75, 80, 85,
    90, 95, 100, 105, 110, 115, 120, 120, 125, 130,
    135, 140, 145, 150, 155, 160, 165, 170, 175, 180,
    185, 190, 195, 200, 205, 210, 215, 220, 225, 230,
    240, 250, 260, 270, 280, 290, 300, 320, 350, 480
]
```

Requirements

Task 1 – Descriptive Statistics

- Calculate mean, median, and mode.
- Find min, max, and range.
- Calculate **sample** variance and **sample** standard deviation.

Task 2 – Frequency Distribution and Histogram

Divide usage into the six groups below:

Group	Usage time (minutes/day)
1	0–60
2	60–120
3	120–180
4	180–240
5	240–360
6	360–500

- Use `pd.cut()` and `value_counts()` to create a frequency distribution table.
- Calculate cumulative frequency using `cumsum()`.

- Create a histogram with title and axis labels.

Task 3 – Quartiles and Outlier Detection

- Calculate Q1, Q2, Q3 and IQR.
- Calculate lower and upper bounds using **1.5 × IQR**.
- Identify usage times outside the bounds.

Task 4 – Sampling and Hypothesis Testing

- Randomly select 10 users.
- Calculate sample mean and compare it with mean of all 50 users.
- Test the company's claim that **population mean usage is 120 minutes/day**, using a **one-sample t-test** at $\alpha = 0.05$.
- Based on p-value, decide whether to reject or fail to reject H_0.

Task 5 – Conclusion

Write a short conclusion (3–5 sentences) about:

- Typical daily app usage time.
- Variability and presence of outliers.
- Hypothesis-test result.
- One recommendation for the company.

Chuyển từ `LAB SESSION 8.docx` sang Markdown.

2. Đề bài tiếng Việt

LAB SESSION 8 — Thống kê

Công ty ứng dụng di động phân tích thời gian sử dụng mỗi ngày của **50 người dùng**, tính bằng **phút/ngày**. Các bài cần dùng NumPy (thư viện tính toán mảng), Pandas (thư viện bảng dữ liệu), SciPy (thư viện tính toán khoa học) và Matplotlib (thư viện vẽ biểu đồ).

Dữ liệu 50 người (phút/ngày)

```
usage_minutes = [
    35, 45, 50, 55, 60, 65, 70, 75, 80, 85,
    90, 95, 100, 105, 110, 115, 120, 120, 125, 130,
    135, 140, 145, 150, 155, 160, 165, 170, 175, 180,
    185, 190, 195, 200, 205, 210, 215, 220, 225, 230,
    240, 250, 260, 270, 280, 290, 300, 320, 350, 480
]
```

Toàn bộ yêu cầu

Bài 1

Từ 50 người dùng và số phút sử dụng ứng dụng mỗi ngày, tính trung bình (mean (trung bình)), trung vị (median (trung vị)), mốt (mode (mốt)), nhỏ nhất, lớn nhất và khoảng biến thiên. Tính **phương sai mẫu** và **độ lệch chuẩn mẫu** (không dùng phương sai tổng thể).

Bài 2

Chia thời gian dùng ứng dụng thành đúng 6 nhóm: 0–60, 60–120, 120–180, 180–240, 240–360, 360–500 phút/ngày. Dùng `pd.cut()` và `value_counts()` tạo bảng tần suất, `cumsum()` tính tần suất tích lũy, vẽ histogram (biểu đồ tần suất) có tiêu đề và nhãn trục. Quy định rõ dữ liệu nằm ở ranh giới 60/120/... được tính đúng một nhóm.

Bài 3

Tính Q1, Q2, Q3 và IQR; xác định biên dưới và biên trên theo quy tắc `1.5 × IQR`, rồi liệt kê các thời gian nằm ngoài hai biên này.

Bài 4

Chọn ngẫu nhiên 10 người dùng, tính trung bình của mẫu 10 và so sánh với trung bình 50 người. Kiểm định giả thuyết công ty rằng trung bình tổng thể là **120 phút/ngày** bằng one-sample (mẫu) t-test, mức ý nghĩa α=0,05. Dựa vào p-value (giá trị xác suất kiểm định) quyết định bác bỏ hay chưa đủ bằng chứng bác bỏ H0.

Bài 5

Viết kết luận ngắn 3–5 câu về mức sử dụng điển hình, độ biến thiên/ngoại lệ, kết quả kiểm định và một khuyến nghị cho công ty. Không mặc định dữ liệu mô phỏng là bằng chứng về toàn bộ người dùng thực.

Bảng nhóm tần suất từ Word: 0–60, 60–120, 120–180, 180–240, 240–360, 360–500 phút/ngày (6 nhóm).

3. Code Python có chú thích

```
from pathlib import Path  # Nạp thư viện (import: nạp vào) để có các hàm đã học.
import sys  # Nạp thư viện (import: nạp vào) để có các hàm đã học.
import numpy as np  # Nạp thư viện (import: nạp vào) để có các hàm đã học.
import pandas as pd  # Nạp thư viện (import: nạp vào) để có các hàm đã học.
from scipy import stats  # Nạp thư viện (import: nạp vào) để có các hàm đã học.

USAGE_MINUTES=np.array([  # Khởi tạo mảng NumPy với số liệu theo đúng để
    35,45,50,55,60,65,70,75,80,85,  # Giữ nguyên giá trị số trong dữ liệu để bài.
    90,95,100,105,110,115,120,120,125,130,  # Giữ nguyên giá trị số trong dữ liệu để bài.
    135,140,145,150,155,160,165,170,175,180,  # Giữ nguyên giá trị số trong dữ liệu để bài.
    185,190,195,200,205,210,215,220,225,230,  # Giữ nguyên giá trị số trong dữ liệu để bài.
    240,250,260,270,280,290,300,320,350,480  # Giữ nguyên giá trị số trong dữ liệu để bài.
],dtype=float)  # Đóng mảng, danh sách hoặc nhóm tham số.

def task01_descriptive(values):  # Định nghĩa hàm (function: hàm) để gom và gọi lại các bước.
    return {  # Trả kết quả (return: trả về) cho nơi gọi hàm.
        "mean":float(values.mean()),"median":float(np.median(values)),  # Gắn khóa (key: tên trường) với giá trị trong từ điển.
        "mode":float(pd.Series(values).mode().iloc[0]),  # Gắn khóa (key: tên trường) với giá trị trong từ điển.
        "minimum":float(values.min()),"maximum":float(values.max()),  # Gắn khóa (key: tên trường) với giá trị trong từ điển.
        "range":float(np.ptp(values)),  # Gắn khóa (key: tên trường) với giá trị trong từ điển.
        "sample_variance":float(values.var(ddof=1)),  # Gắn khóa (key: tên trường) với giá trị trong từ điển.
        "sample_std":float(values.std(ddof=1)),  # Gắn khóa (key: tên trường) với giá trị trong từ điển.
    }  # Đóng mảng, danh sách hoặc nhóm tham số.

def task02_frequency(values):  # Định nghĩa hàm (function: hàm) để gom và gọi lại các bước.
    bins=[0,60,120,180,240,360,500]  # Tính hoặc cập nhật biến bins từ công thức bên phải.
    labels=["0–<60","60–<120","120–<180","180–<240","240–<360","360–<500"]  # Tính hoặc cập nhật biến labels từ công thức bên phải.
    groups=pd.cut(values,bins=bins,labels=labels,right=False,include_lowest=True)  # Tính hoặc cập nhật biến groups từ công thức bên phải.
    freq=pd.Series(groups).value_counts(sort=False)  # Tính hoặc cập nhật biến freq từ công thức bên phải.
    frame=pd.DataFrame({"frequency":freq})  # Tính hoặc cập nhật biến frame từ công thức bên phải.
```

```
    frame["cumulative"]=frame["frequency"].cumsum()  # Tiếp tục bước xử lý dữ liệu đúng như code tham khảo.
    if not (frame["frequency"].sum()==len(values)): raise ValueError("Kiểm tra không đạt")  # Kiểm tra bằng if (nếu), báo lỗi nếu kết quả không phù hợp.
    return bins,frame  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def task03_outliers(values):  # Định nghĩa hàm (function: hàm) để gom và gọi lại các bước.
    q1,q2,q3=np.percentile(values,[25,50,75])  # Tiếp tục bước xử lý dữ liệu đúng như code tham khảo.
    iqr=q3-q1  # Tính hoặc cập nhật biến iqr từ công thức bên phải.
    low,high=q1-1.5*iqr,q3+1.5*iqr  # Tiếp tục bước xử lý dữ liệu đúng như code tham khảo.
    unusual=values[(values<low)|(values>high)]  # Tính hoặc cập nhật biến unusual từ công thức bên phải.
    return q1,q2,q3,iqr,low,high,unusual  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def task04_sample_and_test(values):  # Định nghĩa hàm (function: hàm) để gom và gọi lại các bước.
    rng=np.random.default_rng(2026)  # Rút mẫu ngẫu nhiên; seed (hạt giống) giúp tái lập.
    sample=rng.choice(values,size=10,replace=False)  # Rút mẫu ngẫu nhiên; seed (hạt giống) giúp tái lập.
    result=stats.ttest_1samp(values,popmean=120.0)  # Tính thống kê theo hàm/tham số đã học.
    return sample,float(sample.mean()),float(result.statistic),float(result.pvalue)  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def task05_conclusion(descriptive,outliers,pvalue):  # Định nghĩa hàm (function: hàm) để gom và gọi lại các bước.
    typical=descriptive["median"]  # Tính hoặc cập nhật biến typical từ công thức bên phải.
    decision="reject H0" if pvalue<0.05 else "fail to reject H0"  # Tính hoặc cập nhật biến decision từ công thức bên phải.
    print(f"Typical use is around median {typical:.1f} minutes/day.")  # In kết quả (output: kết quả) để đối chiếu với đề.
    print(f"Sample SD is {descriptive['sample_std']:.1f} minutes, with {len(outliers)} IQR outlier(s).")  # In kết quả (output: kết quả) để đối chiếu với đề.
    print(f"One-sample t-test vs 120 min/day: p={pvalue:.6g}; {decision} at alpha=0.05.")  # In kết quả (output: kết quả) để đối chiếu với đề.
    print("Recommendation: study long-duration sessions separately and design user-friendly break reminders.")  # In kết quả (output: kết quả) để đối chiếu với đề.

def save_plot(values,bins):  # Định nghĩa hàm (function: hàm) để gom và gọi lại các bước.
    import matplotlib  # Nạp thư viện (import: nạp vào) để có các hàm đã học.
    matplotlib.use("Agg")  # Tiếp tục bước xử lý dữ liệu đúng như code tham khảo.
    import matplotlib.pyplot as plt  # Nạp thư viện (import: nạp vào) để có các hàm đã học.
    output=Path(__file__).resolve().parent/"outputs"  # Tính hoặc cập nhật biến output từ công thức bên phải.
    output.mkdir(exist_ok=True)  # Tiếp tục bước xử lý dữ liệu đúng như code tham khảo.
    fig,ax=plt.subplots()  # Tiếp tục bước xử lý dữ liệu đúng như code tham khảo.
    ax.hist(values,bins=bins,edgecolor="black")  # Tiếp tục bước xử lý dữ liệu đúng như code tham khảo.
    ax.set(xlabel="Daily app usage (minutes)",ylabel="Users",title="Daily usage of 50 users")  # Đặt nhãn, lưới hoặc chú giải cho biểu đồ.
    ax.grid(True,axis="y")  # Đặt nhãn, lưới hoặc chú giải cho biểu đồ.
    target=output/"usage_histogram.png"  # Tính hoặc cập nhật biến target từ công thức bên phải.
    fig.savefig(target,dpi=140,bbox_inches="tight")  # Vẽ/lưu biểu đồ`(plot: đồ`thị) để diễn giải kết quả.
    plt.close(fig)  # Đóng biểu đồ`để tránh giữ tài nguyên.
    print("Saved:",target)  # In kết quả (output: kết quả) để đối chiếu với đề.

def main():  # Định nghĩa hàm (function: hàm) để gom và gọi lại các bước.
    if not (USAGE_MINUTES.shape==(50,)): raise ValueError("Kiểm tra không đạt")  # Kiểm tra bằng if (nếu), báo lỗi nếu kết quả không phù hợp.
    desc=task01_descriptive(USAGE_MINUTES)  # Tính hoặc cập nhật biến desc từ công thức bên phải.
    bins,freq=task02_frequency(USAGE_MINUTES)  # Tiếp tục bước xử lý dữ liệu đúng như code tham khảo.
    q1,q2,q3,iqr,lo,hi,outliers=task03_outliers(USAGE_MINUTES)  # Tiếp tục bước xử lý dữ liệu đúng như code tham khảo.
    sample,sample_mean,t,p=task04_sample_and_test(USAGE_MINUTES)  # Tiếp tục bước xử lý dữ liệu đúng như code tham khảo.
    if not (len(sample)==10): raise ValueError("Kiểm tra không đạt")  # Kiểm tra bằng if (nếu), báo lỗi nếu kết quả không phù hợp.
    if not (0<=p<=1): raise ValueError("Kiểm tra không đạt")  # Kiểm tra bằng if (nếu), báo lỗi nếu kết quả không phù hợp.
    print("Descriptive:",desc)  # In kết quả (output: kết quả) để đối chiếu với đề.
    print("Frequency:\n",freq.to_string())  # In kết quả (output: kết quả) để đối chiếu với đề.
    print(f"Q1={q1:.2f}, Q2={q2:.2f}, Q3={q3:.2f}, IQR={iqr:.2f}")  # In kết quả (output: kết quả) để đối chiếu với đề.
    print(f"Outlier limits [{lo:.2f},{hi:.2f}], values:",outliers.tolist())  # In kết quả (output: kết quả) để đối chiếu với đề.
    print("Sample of 10:",sample.tolist(),"sample mean:",round(sample_mean,2))  # In kết quả (output: kết quả) để đối chiếu với đề.
    print("t:",round(t,4),"p:",round(p,6))  # In kết quả (output: kết quả) để đối chiếu với đề.
    task05_conclusion(desc,outliers,p)  # Tiếp tục bước xử lý dữ liệu đúng như code tham khảo.
    if "--no-plot" not in sys.argv:  # Kiểm tra điều kiện (condition: điều kiện), thực hiện nhánh khi đúng.
        save_plot(USAGE_MINUTES,bins)  # Tiếp tục bước xử lý dữ liệu đúng như code tham khảo.
```

```
if __name__=="__main__":  # Kiểm tra điều kiện (condition: điều kiện), thực hiện nhánh khi đúng.
    main()  # Tiếp tục bước xử lý dữ liệu đúng như code tham khảo.
```

4. Giải thích nội dung và cách giải

Câu 1

Dữ liệu 50 dòng được đưa nguyên vào `USAGE_MINUTES`. Dùng NumPy (thư viện tính toán mảng)/Pandas (thư viện bảng dữ liệu) tính các chỉ số; dùng `ddof=1` để phương sai chia cho n−1 chứ không phải n. Mode (mốt) được lấy bằng `.mode().iloc[0]`.

Vì sao cần các chỉ số khác nhau? Trung bình chỉ mức sử dụng chung nhưng dễ bị kéo lên bởi 480 phút; trung vị cho mức giữa; phương sai và độ lệch chuẩn cho thấy độ phân tán. Vì đang mô tả một mẫu 50 người, lời giải dùng `ddof=1` cho thống kê mẫu.

Các bước: Chuyển 50 số phút sang mảng số; tính trung bình, trung vị, mốt, min/max; tính khoảng biến thiên, phương sai mẫu và độ lệch chuẩn mẫu rồi so sánh.

```
import numpy as np
import pandas as pd
values = USAGE_MINUTES.astype(float)  # Dùng dữ liệu đúng như code hoàn chỉnh cuối bài
mean = values.mean()
median = np.median(values)
mode = pd.Series(values).mode().iloc[0]
spread = np.ptp(values)
variance = values.var(ddof=1)
std = values.std(ddof=1)
```

Tham số và cú pháp: `dtype=float` chọn kiểu số thực; `np.median` tính trung vị; `.mode()` trả một tập các giá trị có tần số cao nhất, `.iloc[0]` chọn giá trị đầu tiên nếu có nhiều mốt; `np.ptp` tính max trừ min. `ddof=1` nghĩa là hiệu chỉnh mẫu: chia cho `n-1` thay vì `n`. Đơn vị của `variance` là phút bình phương, còn `std` là phút.

Ví dụ/lỗi thường gặp: Giữ đủ 50 số phút, kể cả 480. Phương sai và độ lệch chuẩn **mẫu** chia cho n−1 (ddof=1), không chia n như tổng thể.

Từ vựng chính: Descriptive statistics (thống kê mô tả).

Câu 2

Các khoảng liên tiếp trong `bins=[0,60,120,180,240,360,500]`, `right=False` nghĩa là [0,60), [60,120),... Tránh một phút đúng 60 hoặc 120 bị tính hai lần. Lấy tần số của từng khoảng rồi cộng dồn; hình histogram (biểu đồ tần suất) dùng chức năng `save_plot` đã có.

Vì sao gom theo khoảng? Bảng có 50 giá trị riêng lẻ khó thấy mức sử dụng nào phổ biến. Chia vào cùng các khoảng phút giúp đếm người theo nhóm; tần số tích lũy trả lời đã có bao nhiêu người ở các nhóm từ đầu tới nhóm hiện tại.

Các bước: (1) Định nghĩa biên nhóm theo phút. (2) Phân từng thời gian vào một nhóm. (3) Đếm tần số theo thứ tự nhóm, không đảo theo nhóm phổ biến. (4) Cộng dồn và kiểm tra tổng là 50.

```
import pandas as pd
bins = [0, 60, 120, 180, 240, 360, 500]
labels = ["0–<60", "60–<120", "120–<180", "180–<240", "240–<360", "360–<500"]
groups = pd.cut(values, bins=bins, labels=labels, right=False, include_lowest=True)
freq = pd.Series(groups).value_counts(sort=False)
cumulative = freq.cumsum()
```

Flag và tham số: `bins` là các ranh giới nhóm; `labels` là tên nhóm; `right=False` dùng khoảng gồm cận trái nhưng **không** gồm cận phải, nên 120 được xếp nhóm 120–180; `include_lowest=True` không bỏ cận thấp nhất. `value_counts(sort=False)` giữ thứ tự nhóm thay vì sắp theo lượng người. `cumsum()` cộng dồn. Đồ thị `ax.hist(values, bins=bins)` phải dùng cùng ranh giới nhóm.

Ví dụ/lỗi thường gặp: Nhóm khoảng theo [0,60), [60,120)... để số 60 không bị đếm hai lần. Tổng tần số phải bằng 50.

Từ vựng chính: Histogram (biểu đồ tần suất).

Câu 3

`np.percentile` lấy ba phân vị, dùng IQR làm thước đo độ trải rộng ít nhạy với ngoại lệ. Các điểm nhỏ hơn biên dưới hoặc lớn hơn biên trên được đánh dấu, không tự động xóa.

Vì sao dùng IQR? Giá trị 480 phút có thể khác thường. Dùng khoảng tứ phân vị giúp xác định điểm ngoài vùng dữ liệu trung tâm mà không tự ý xóa hoặc sửa dữ liệu nguồn.

Các bước: (1) Tính tứ phân vị 25%, 50%, 75%. (2) Lấy Q3 trừ Q1 để tính IQR. (3) Tính biên dưới và biên trên bằng 1.5 lần IQR. (4) Lọc các giá trị nhỏ hơn biên dưới hoặc lớn hơn biên trên.

```
q1, median, q3 = np.percentile(values, [25, 50, 75])
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
outliers = values[(values < lower) | (values > upper)]
```

Tham số và toán tử: `np.percentile(values, [25, 50, 75])` trả ba mốc phần trăm; `|` kết hợp hai mảng điều kiện theo nghĩa **hoặc**; dấu ngoặc quanh mỗi phép so sánh là cần thiết. `1.5` là hệ số quy ước của cách phát hiện ngoại lệ IQR, **không** phải mức ý nghĩa thống kê. Ngoại lệ được đánh dấu để phân tích, không tự động loại bỏ.

Ví dụ/lỗi thường gặp: IQR=Q3−Q1; biên dưới=Q1−1,5×IQR và biên trên=Q3+1,5×IQR. Không tự xóa ngoại lệ trước khi tính.

Từ vựng chính: Quartiles (tứ phân vị).

Câu 4

`default_rng(2026).choice(...,size=10,replace=False)` chọn 10 giá trị không lặp. Code kiểm định đã có gọi `ttest_1samp(values,popmean=120)`, tức dùng **toàn bộ 50 quan sát** cho phép kiểm định, còn mẫu 10 chỉ minh họa lấy mẫu. Đừng gọi nhầm t-test chỉ trên 10 quan sát. So sánh p-value (giá trị xác suất kiểm định) với 0,05.

Vì sao có hai bước lấy mẫu và kiểm định? Lấy ngẫu nhiên 10 người minh họa sampling (lấy mẫu), còn giả thuyết phải so sánh mức trung bình với mốc 120 phút. Code tham khảo thực hiện kiểm định t trên **đủ 50 giá trị**, không nhầm dùng 10 mẫu vừa rút để kiểm định.

Các bước: (1) Dùng hạt giống cố định rút 10 người không hoàn lại. (2) Lấy trung bình của 10 người để so sánh. (3) Kiểm định t một mẫu với mốc giả thuyết 120 phút bằng 50 giá trị. (4) So `pvalue` với ngưỡng 0.05.

```
from scipy import stats
rng = np.random.default_rng(2026)
sample = rng.choice(values, size=10, replace=False)
sample_mean = sample.mean()
result = stats.ttest_1samp(values, popmean=120.0)
t_value, p_value = result.statistic, result.pvalue
reject_h0 = p_value < 0.05
```

Flag và tham số: `size=10` là số người rút; `replace=False` nghĩa là **không hoàn lại** nên không chọn một người hai lần; `popmean=120.0` là giá trị trung bình giả định cần kiểm định. `result.statistic` là thống kê t; `result.pvalue` là p-value. `p_value < 0.05` cho biết bác bỏ giả thuyết gốc ở mức ý nghĩa 5% nếu đúng; nếu sai thì **chưa đủ bằng chứng** bác bỏ, không khẳng định giả thuyết gốc là đúng. Kiểm định t cần xem xét giả định thống kê về dữ liệu.

Ví dụ/lỗi thường gặp: Lấy 10 mẫu không hoàn lại, nhưng t-test trong code nguồn dùng 50 quan sát. So sánh p-value (giá trị xác suất kiểm định) với 0,05 trước khi kết luận.

Từ vựng chính: Hypothesis testing (kiểm định giả thuyết).

Câu 5

Hàm `task05_conclusion` của code tham khảo in bốn ý: dùng median (trung vị) đại diện mức điển hình, SD và số điểm ngoại lệ để mô tả độ biến thiên, đọc p-value (giá trị xác suất kiểm định) cho kiểm định, và đề xuất xem xét phiên dùng dài cùng lời nhắc nghỉ. Điều này đáp ứng 3–5 câu của đề.

Vì sao kết luận dựa trên nhiều chỉ số? Chỉ nhìn trung bình có thể bỏ qua độ phân tán và người dùng bất thường. Một khuyến nghị cần dựa trên trung vị, độ lệch chuẩn, ngoại lệ và kết quả kiểm định đã có, không suy diễn tác động nhân quả từ dữ liệu quan sát.

Các bước: (1) Chọn trung vị làm đại diện cho mức dùng thường gặp. (2) Nêu độ lệch chuẩn mẫu và số ngoại lệ IQR. (3) Diễn giải ý nghĩa kiểm định t. (4) Đề xuất quan sát các phiên dùng dài và nhắc nghỉ hợp lý.

```
median_minutes = descriptive["median"]
sample_std = descriptive["sample_std"]
outlier_count = len(outliers)
decision = "bác bỏ H0" if pvalue < 0.05 else "chưa bác bỏ H0"
summary = (f"Trung vị {median_minutes:.1f} phút; "
           f"SD mẫu {sample_std:.1f}; {outlier_count} ngoại lệ; {decision}.")
```

Cú pháp và tham số: `descriptive["median"]` lấy giá trị theo tên trường của dictionary; `len(outliers)` đếm ngoại lệ. Biểu thức `A if điều_kiện else B` chọn câu giải thích đúng theo p-value; `:.1f` trong f-string chỉ quy định **một chữ số thập phân khi in**, không thay số đã tính. Không coi ngoại lệ 480 phút là lỗi dữ liệu chỉ vì giá trị lớn.

Ví dụ/lỗi thường gặp: Đối chiếu median (trung vị), SD, ngoại lệ và kiểm định khi viết 3–5 câu. Không suy rộng số liệu mô phỏng thành kết luận cho mọi người dùng.

Từ vựng chính: Conclusion (kết luận).